

Software Guide

Data Structures, Algorithms, and Applications in C++

76 header-only implementations · dsa namespace · C++23

Part II — Data Structures

Linear Lists

Array List

Header: code/include/array_list.h

Class: dsa::array_list<T>

Dynamic array with amortized $O(1)$ append. Grows via reallocation. This is the most fundamental sequential container, analogous to `std::vector` but built from scratch for pedagogical clarity.

Why: You need fast random access and appending, and don't need insertion/deletion in the middle.

Where to use: Use when building other data structures (stacks, queues, deques), managing collections with index access, or as scratch buffers. Default choice for sequential data.

Input: Elements of type T (must satisfy `std::regular`)

Output: Index-based access via `operator[]`; iterators for traversal

```
dsa::array_list<int> a;  
a.push_back(10);  
a.push_back(20);  
a[0]; // 10  
a.size(); // 2
```

Linked List

Header: code/include/linked_list.h

Class: dsa::linked_list<T>

Singly linked list using `std::unique_ptr` for automatic memory management. Supports forward and reverse iterators.

Why: You need $O(1)$ insert/delete at known positions without shifting elements, and don't need random access.

Where to use: Use for adjacency lists in sparse graphs, undo/redo buffers, implementing other containers, and when elements are frequently inserted/deleted in the middle.

Input: Elements of any copyable/movable type

Output: $O(1)$ insert at front; iteration via range-for

```
dsa::linked_list<int> l;
```

```
l.push_front(3);
l.push_back(4);
for (auto x : l) { /* 3, 4 */ }
```

Doubly Linked List

Header: code/include/doubly_linked_list.h

Class: dsa::doubly_linked_list<T>

Doubly linked list with sentinel node for clean boundary handling. Bidirectional traversal.

Why: You need $O(1)$ insert/delete at both ends AND $O(1)$ delete at arbitrary positions (given iterator).

Where to use: Use for LRU caches (with hash map), browser history, text editor buffers, and deques. Better than singly linked list when you need backward traversal.

```
dsa::doubly_linked_list<int> dll;
dll.push_back(1);
dll.push_back(2);
dll.push_front(0);
dll.front(); // 0
dll.back();  // 2
dll.pop_back();
for (auto x : dll) { /* 0, 1 */ }
```

Bit Vector

Header: code/include/bit_vector.h

Class: dsa::bit_vector

Compact bit storage with bulk operations. Uses native word-level parallelism via std::popcount.

Why: You need compact storage for boolean/bit flags with fast bulk operations (popcount, find-first).

Where to use: Use for visited arrays in BFS/DFS, bloom filter internals, database bitmap indexes, permission flags, and any compact boolean set over a small universe.

```
dsa::BitVector bv(8);
bv.set(3); bv.set(5);
bv.count(); // 2
bv.find_first(); // 3
bv.flip(3);
bv.get(3); // false
```

Sparse Matrix

Header: code/include/sparse_matrix.h

Class: dsa::sparse_matrix

Sparse matrix in COO (coordinate) and CSR (compressed sparse row) format. Supports transpose and matrix-vector multiply.

Why: Your matrix is mostly zeros and storing it densely wastes memory and compute.

Where to use: Use for large-scale scientific computing, ML feature matrices, graph adjacency matrices, and FEM/PDE solvers where $\text{nnz} \ll \text{rows} \times \text{cols}$.

```
dsa::SparseMatrix<double> m(3, 3);
m.set(0, 0, 1.0);
m.set(1, 2, 3.0);
auto t = m.transpose();
std::vector<double> x = {1, 2, 3};
auto y = m.multiply(x); // matrix-vector product
```

Sparse Vector

Header: code/include/sparse_vector.h

Class: dsa::sparse_vector

Why: Your vector is high-dimensional but mostly zeros, and dense storage wastes memory.

Where to use: Use for TF-IDF vectors in IR, ML feature representations, recommendation system embeddings, and any sparse high-dimensional data.

```
dsa::SparseVector<double> sv(100);
sv.set(5, 2.0);
sv.set(99, 3.0);
double d = sv.dot(sv); // 13.0
auto s = sv.scale(2.0);
```

Stacks

Header: code/include/stacks.h

Classes: dsa::array_stack<T>, dsa::linked_stack<T>

Two implementations: vector-backed array stack and pointer-based linked stack. LIFO access pattern.

Why: You need LIFO access: last-in, first-out.

Where to use: Use for expression evaluation, DFS traversal, backtracking, undo systems, parenthesis matching, and converting infix to postfix notation.

```
dsa::array_stack<int> s;
s.push(1);
s.push(2);
s.top(); // 2
s.pop();
s.top(); // 1
```

Queues & Deques

Header: code/include/queues.h

Classes: dsa::circular_queue<T>, dsa::linked_queue<T>

FIFO access pattern. Circular queue uses a fixed-size ring buffer for predictable performance.

Why: You need FIFO access: first-in, first-out.

Where to use: Use for BFS traversal, task scheduling, print spooling, producer-consumer patterns, and request handling in servers.

Deque

Header: code/include/deque.h

Class: dsa::deque<T>

Double-ended queue (block-based circular buffer). Combines the benefits of arrays and linked lists.

Why: You need $O(1)$ push/pop at both ends AND $O(1)$ random access by index.

Where to use: Use for sliding window algorithms, BFS with deques, implementing std::deque, and when you need both stack and queue behavior.

```
dsa::Deque<int> dq;
dq.push_back(1);
dq.push_back(2);
dq.push_front(0);
dq[0];    // 0
dq.size(); // 3
dq.pop_front();
```

Hashing

Hash Table (Chaining)

Header: code/include/hash_table.h

Class: dsa::hash_table_chaining<K,V>

Hash table with separate chaining. Each bucket is a linked list handling collisions gracefully. Supports iterators for traversal.

Why: You need $O(1)$ average-case insert/find/erase by key, and don't need ordering.

Where to use: Use for symbol tables, caches, frequency counting, database indexes, and any key-value lookup where order doesn't matter.

```
dsa::hash_table_chaining<std::string,int> h;
h.insert("one", 1);
h.find("one"); // 1
h.erase("one");
```

Open Addressing Hash

Header: code/include/open_addressing_hash.h

Classes: linear probing, quadratic probing, double hashing

All elements stored in a single array. Three probing strategies for collision resolution. Better cache performance than chaining when load factor is moderate.

Why: You need a hash table with better cache locality than chaining, and can tolerate slightly higher collision costs.

Where to use: Use when memory is constrained (no pointer overhead), for cache-friendly hash maps, and in real-time systems. Linear probing is simplest; double hashing gives best distribution.

```
dsa::open_addressing_hash<std::string,int> h;
h.insert("one", 1);
h.insert("two", 2);
auto* val = h.find("one"); // points to 1
h.erase("two");
h.contains("two"); // false
```

Skip List

Header: code/include/skip_list.h

Class: dsa::skip_list<T>

Randomized ordered structure providing balanced BST performance without rotations. Multiple levels of forward pointers give $O(\log n)$ expected time.

Why: You need $O(\log n)$ ordered operations with simple implementation, and can accept probabilistic balance.

Where to use: Use as a simpler alternative to balanced BSTs when exact balance isn't critical. Great for concurrent access (lock-free insert), databases, and when you want expected $O(\log n)$ without complex rotations.

```
dsa::skip_list<int> sl;
sl.insert(5);
sl.insert(3);
sl.insert(7);
sl.contains(5); // true
sl.erase(3);
auto v = sl.to_sorted_vector(); // [5, 7]
```

Trees

Binary Tree

Header: code/include/binary_tree.h

Class: dsa::binary_tree<T>

Full binary tree with four traversals (inorder, preorder, postorder, level-order), Morris traversal ($O(1)$ extra space), and BST insert/erase/find.

Why: You need a general tree structure for hierarchical data or as a foundation for other algorithms.

Where to use: Use as the building block for BSTs, heaps, Huffman trees, expression parsing, and decision trees. Start here before moving to balanced variants.

```
dsa::binary_tree<int> bt;
bt.insert(5);
bt.insert(3);
bt.insert(7);
bt.contains(3); // true
bt.erase(3);
```

Expression Tree

Header: code/include/expression_tree.h

Class: dsa::expression_tree

Build from postfix notation, evaluate arithmetic expressions, convert between infix/postfix/preorder notations.

Why: You need to parse, evaluate, or transform arithmetic expressions.

Where to use: Use in compilers, calculators, symbolic math systems, and query optimizers for expression evaluation and notation conversion.

```
std::vector<std::string> postfix = {"3", "4", "+", "2", "*"};
auto* root = dsa::build_expr_tree(postfix);
double val = dsa::eval_expr_tree(root); // 14.0
```

Heaps & Priority Queues

Max Heap

Header: code/include/max_heap.h

Class: dsa::max_heap<T>

Standard binary max-heap implemented as an array. Also provides `heap_sort()` for in-place $O(n \log n)$ sorting.

Why: You need fast access to the maximum element with $O(\log n)$ insert/delete.

Where to use: Use as the default priority queue, for heap sort, finding k-th largest, scheduling, and Dijkstra's algorithm. Also useful for median maintenance with two heaps.

```
dsa::max_heap<int> h;
h.push(3); h.push(1); h.push(4);
h.top(); // 4
h.pop();
h.top(); // 3
std::vector<int> v = {5, 1, 3, 2};
```

```
dsa::heap_sort(v); // v = {1, 2, 3, 5}
```

D-ary Heap

Header: code/include/dary_heap.h

Class: dsa::dary_heap<T,D>

Generalized d-children heap (default D=4). Higher branching factor improves cache performance for decrease-key operations.

Why: You need a priority queue with many decrease-key operations and better cache performance than binary heap.

Where to use: Use for Dijkstra's and Prim's algorithms on dense graphs where cache performance matters more than theoretical optimality.

```
dsa::dary_heap<int, 4> h;  
h.push(3); h.push(1); h.push(4);  
h.top(); // 1 (min-heap)  
h.pop();
```

Indexed Priority Queue

Header: code/include/indexed_pq.h

Classes: dsa::indexed_min_pq<T>, dsa::indexed_max_pq<T>

Priority queue that supports updating priorities by index. Tracks which element is at each position.

Why: You need a priority queue where you can update the priority of any element by its index.

Where to use: Use for Dijkstra's, Prim's, A* search, and any algorithm requiring decrease-key or increase-key operations on specific elements.

```
dsa::indexed_min_pq<int> pq(10);  
pq.push(0, 5);  
pq.push(1, 3);  
pq.push(2, 7);  
pq.decrease_key(2, 1);  
int idx = pq.pop(); // 2 (min key = 1)
```

Min-Max Heap

Header: code/include/min_max_heap.h

Class: dsa::min_max_heap<T>

Double-ended priority queue. Supports efficient access to both minimum and maximum elements simultaneously.

Why: You need O(1) access to BOTH minimum and maximum simultaneously.

Where to use: Use for median maintenance, sliding window min/max, and double-ended priority queues. Better than two separate heaps when you need both ends.

```
dsa::min_max_heap<int> mmh;  
mmh.push(3); mmh.push(1); mmh.push(5);  
mmh.min(); // 1  
mmh.max(); // 5  
mmh.pop_min(); // removes 1
```

Interval Heap

Header: code/include/interval_heap.h

Class: dsa::interval_heap

Double-ended priority queue using two sorted sets, storing interval pairs (lo, hi).

Why: You need a double-ended priority queue that stores interval pairs [lo, hi].

Where to use: Use for interval scheduling, deadline management, and event processing where you track both start and end times.

```
dsa::interval_heap ih;  
ih.push(2, 8); // interval [2,8]  
ih.push(1, 5); // interval [1,5]  
ih.min(); // 1  
ih.max(); // 8
```

Binomial Heap

Header: code/include/binomial_heap.h

Class: dsa::binomial_heap<T>

Mergeable heap using binomial trees. Efficient meld (merge) operation unlike binary heaps.

Why: You need $O(\log n)$ amortized merge of two priority queues, which binary heaps can't do efficiently.

Where to use: Use for k-way merge in external sorting, Dijkstra's/Prim's on dense graphs, and when you frequently merge two priority queues.

```
dsa::BinomialHeap<int> h1, h2;  
h1.push(3); h1.push(1);  
h2.push(2);  
h1.merge(h2);  
int top = h1.pop(); // 1
```

Fibonacci Heap

Header: code/include/fibonacci_heap.h

Class: dsa::fibonacci_heap<T>

Theoretically optimal decrease-key performance. Amortized $O(1)$ for insert and decrease-key, $O(\log n)$ for delete-min.

Why: You need theoretically optimal $O(1)$ amortized decrease-key for dense graph algorithms.

Where to use: Use when theoretical optimality matters: Dijkstra's in $O(E + V \log V)$, Prim's, and computational geometry. Rarely used in practice due to high constant factors.

```
dsa::FibonacciHeap<int> fh;
fh.push(5); fh.push(3); fh.push(7);
int top = fh.pop(); // 3
fh.push(1);
top = fh.pop(); // 1
```

Pairing Heap

Header: code/include/pairing_heap.h

Class: dsa::pairing_heap<T>

Simple mergeable heap with two-pass meld. Simpler than Fibonacci heaps with comparable practical performance.

Why: You want Fibonacci-heap-like performance with simpler implementation.

Where to use: Use as a practical alternative to Fibonacci heaps for Dijkstra's, event simulations, and whenever you need merge + decrease-key without the complexity.

```
dsa::PairingHeap<int> ph;
ph.push(3); ph.push(1); ph.push(4);
int top = ph.pop(); // 1
```

Skew Heap

Header: code/include/skew_heap.h

Class: dsa::skew_heap<T>

Self-adjusting mergeable heap. No balance information stored; structure adapts through rotations on every operation.

Why: You need a mergeable heap with self-adjusting structure and no balance overhead.

Where to use: Use for mergeable priority queues when simplicity matters more than worst-case guarantees. Good for shortest path algorithms.

```
dsa::skew_heap<int> sh1, sh2;
sh1.push(3); sh1.push(1);
sh2.push(2);
sh1.meld(std::move(sh2));
int top = sh1.pop(); // 1
```

Leftist Tree

Header: code/include/leftist_tree.h

Class: dsa::leftist_tree<T>

Mergeable min-heap with the leftist property: the null-path length of the left child is always $\geq$ the right child's.

Why: You need $O(\log n)$ merge with the leftist property guaranteeing shortest right path.

Where to use: Use for k-way merge in external sorting, tournament trees, and when you need guaranteed $O(\log n)$ merge with simpler code than binomial/Fibonacci heaps.

```
dsa::leftist_tree<int> lt1, lt2;
lt1.push(3); lt1.push(1);
lt2.push(2);
lt1.meld(std::move(lt2));
auto sorted = lt1.drain_sorted(); // [1, 2, 3]
```

Balanced Search Trees

AVL Tree

Header: code/include/avl_tree.h

Class: dsa::avl_tree<K,V>

Height-balanced BST. Balance factor for every node is in $\{-1, 0, 1\}$. Strictly balanced, giving optimal lookup time.

Why: You need strictly balanced BST with optimal lookup time and can tolerate more rotations on insert/erase.

Where to use: Use when lookups dominate over insertions. Stricter balance than red-black trees gives faster search but more rotation overhead on updates.

```
dsa::avl_tree<int,std::string> t;
t.insert(5, "five");
t.insert(3, "three");
t.find(5); // "five"
t.inorder(); // [(3,"three"),(5,"five")]
```

Red-Black Tree

Header: code/include/red_black_tree.h

Class: dsa::red_black_tree<K,V>

2-3-4 tree isomorphic. Less rigidly balanced than AVL but fewer rotations on insert/erase. This is the basis for `std::map` and `std::set`.

Why: You need a general-purpose ordered map with $O(\log n)$ all operations and fewer rotations than AVL.

Where to use: Use as the default ordered map/set. Fewer rotations than AVL on insert/erase make it better for write-heavy workloads. This is what `std::map` uses.

```
dsa::RedBlackTree<int> rbt;
rbt.insert(5); rbt.insert(3); rbt.insert(7);
rbt.contains(5); // true
rbt.erase(3);
```

```
auto v = rbt.inorder(); // [5, 7]
```

Splay Tree

Header: code/include/splay_tree.h

Class: dsa::splay_tree<K,V>

Self-adjusting BST. Every access splays the element to the root via rotations, exploiting access locality.

Why: Your workload has access locality (recently accessed items are accessed again soon).

Where to use: Use when access patterns have temporal locality. Splaying brings frequently accessed elements near root, giving amortized $O(\log n)$ with good cache behavior.

```
dsa::SplayTree<int> st;
st.insert(5); st.insert(3); st.insert(7);
st.contains(5); // true, splays 5 to root
st.erase(3);
```

Treap

Header: code/include/treap.h

Class: dsa::treap<K,V>

BST ordered by key, heap-ordered by random priority. The randomization eliminates adversarial input. Supports split() and merge() for set operations.

Why: You need a balanced BST with $O(\log n)$ split/merge operations for set operations.

Where to use: Use for implicit treaps (sequence manipulation), order-statistics, split/merge set operations, and when you want expected $O(\log n)$ without worst-case balance.

```
dsa::treap<int> t;
t.insert(5); t.insert(3); t.insert(7);
t.contains(5); // true
auto v = t.inorder(); // [3, 5, 7]
t.erase(3);
```

2-3 Tree

Header: code/include/two_three_tree.h

Class: dsa::two_three_tree<K>

All leaves at same depth. Nodes hold 1-2 keys and have 2-3 children. Conceptually simpler than red-black trees.

Why: You want the simplest B-tree variant for learning or for small datasets where disk I/O isn't the bottleneck.

Where to use: Use as an educational tool for understanding B-trees and disk-based indexing. Good for small in-memory datasets.

```
dsa::TwoThreeTree<int> t23;  
t23.insert(5); t23.insert(3); t23.insert(7);  
t23.contains(5); // true  
t23.erase(3);  
auto v = t23.inorder(); // [5, 7]
```

2-3-4 Tree

Header: code/include/two_three_four_tree.h

Class: dsa::two_three_four_tree<K>

B-tree of order 4. Isomorphic to red-black trees. Every internal node has 2-4 children.

Why: You want a B-tree of order 4 with simpler insertion than red-black trees.

Where to use: Use for database indexing and file systems. Isomorphic to red-black trees but with simpler top-down insertion logic.

```
dsa::two_three_four_tree<int, std::string> t234;  
t234.insert(5, "five"); t234.insert(3, "three");  
auto* val = t234.find(5); // "five"  
t234.erase(3);
```

B-Tree

Header: code/include/b_tree.h

Class: dsa::b_tree<K,V,MaxKeys>

Generalized balanced search tree tuned for disk I/O. Large branching factor minimizes disk reads.

Why: You need a disk-friendly tree that minimizes I/O by maximizing branching factor.

Where to use: Use for disk-based indexing where each node fits a disk page. Large branching factor (100s of keys per node) minimizes disk reads.

```
dsa::b_tree<int> bt(5); // order 5  
bt.insert(5); bt.insert(3); bt.insert(7);  
bt.contains(5); // true  
auto v = bt.inorder(); // [3, 5, 7]
```

B+ Tree

Header: code/include/bplus_tree.h

Class: dsa::bplus_tree<K,V>

All data in leaves; leaves linked for efficient range scans. The workhorse of database indexing.

Why: You need both point lookups AND efficient range queries on sorted data.

Where to use: Use when your workload combines point lookups with range scans. Leaves are linked for $O(k + \log n)$ range queries. The standard for database indexes.

```
dsa::bplus_tree<int, std::string> bpt;
bpt.insert(5, "five"); bpt.insert(3, "three");
auto* val = bpt.find(5); // "five"
auto range = bpt.range(1, 6); // [(3,"three"),(5,"five")]
```

(a,b)-Tree

Header: code/include/ab_tree.h

Class: dsa::ab_tree<K>

B-tree generalization with configurable min/max children bounds a and b . Provides fine-grained control over node sizes.

Why: You need fine-grained control over node sizes with configurable min/max children bounds.

Where to use: Use when you want to tune the B-tree family parameters for specific page sizes or memory layouts.

```
dsa::ab_tree<int, std::string, 2, 4> abt;
abt.insert(5, "five"); abt.insert(3, "three");
auto* val = abt.find(5); // "five"
```

Scapegoat Tree

Header: code/include/scapegoat_tree.h

Class: dsa::scapegoat_tree<K>

Weight-balanced BST. Rebuilds an entire subtree when balance exceeds $\alpha = 0.75$. No balance info stored per node.

Why: You need a self-balancing BST with zero per-node balance metadata.

Where to use: Use when memory is tight and you can't afford per-node balance fields. Rebuilds subtrees instead of tracking balance.

```
dsa::scapegoat_tree<int, std::string> sgt;
sgt.insert(5, "five"); sgt.insert(3, "three");
auto* val = sgt.find(5); // "five"
sgt.erase(3);
```

AA Tree

Header: code/include/aa_tree.h

Class: dsa::aa_tree<K,V>

Simplified red-black tree variant where all red links are right links. Only skew and split operations needed.

Why: You want the simplest self-balancing BST with only skew and split operations.

Where to use: Use when you want a simpler red-black tree implementation. Only two operations (skew + split) make it easier to teach and implement correctly.

```
dsa::aa_tree<int, std::string> aat;
aat.insert(5, "five"); aat.insert(3, "three");
auto* val = aat.find(5); // "five"
aat.erase(3);
```

Left-Leaning Red-Black Tree

Header: code/include/llrb_tree.h

Class: dsa::llrb_tree<K,V>

Sedgewick's 2008 variant. Enforces that all red links lean left, reducing the number of cases in rotations.

Why: You want a red-black tree variant with fewer rotation cases by enforcing left-leaning red links.

Where to use: Use when you want a cleaner red-black tree implementation. Sedgewick's variant reduces rotation cases from 7 to 2.

```
dsa::llrb_tree<int, std::string> llrb;
llrb.insert(5, "five"); llrb.insert(3, "three");
auto* val = llrb.find(5); // "five"
llrb.erase(3);
```

Radix/Patricia Trie

Header: code/include/radix_tree.h

Class: dsa::radix_trie

Compressed trie with merged single-child edges. Edges carry string labels instead of single characters.

Why: Your keys share common prefixes and you need efficient prefix-based operations.

Where to use: Use for IP routing (longest prefix match), autocomplete, spell-checking, and any prefix-based dictionary. Compressed trie merges single-child edges.

```
dsa::RadixTree<int> rt;
rt.insert("hello", 1); rt.insert("help", 2);
auto* val = rt.find("hello"); // 1
auto matches = rt.prefix_search("hel"); // [{"hello",1}, {"help",2}]
```

Suffix Tree

Header: code/include/suffix_tree.h

Class: dsa::suffix_tree

Naive construction for all suffixes of a string. Enables linear-time pattern matching after preprocessing.

Why: You need to answer multiple substring queries on the same text after preprocessing.

Where to use: Use for longest common substring, repeated substring detection, and pattern matching when you'll query the same text many times.

```
dsa::SuffixTree st("bananas");
auto pos = st.search("ana"); // {1, 3}
auto lrs = st.longest_repeated_substring(); // "ana"
```

Suffix Array

Header: code/include/suffix_array.h

Class: dsa::suffix_array

Space-efficient alternative to suffix arrays. The sorted array of all suffixes plus LCP (longest common prefix) array.

Why: You need space-efficient full-text indexing with pattern search capability.

Where to use: Use when memory is a concern: suffix arrays use $O(n)$ space vs $O(n^2)$ for suffix trees. Good for full-text search and bioinformatics.

```
dsa::SuffixArray sa("banana");
auto pos = sa.search("ana"); // {1, 3}
auto count = sa.count_occurrences("ana"); // 2
```

Interval Tree

Header: code/include/interval_tree.h

Class: dsa::interval_tree

Augmented BST for interval stabbing queries. Each node stores the maximum endpoint in its subtree.

Why: You need to find all intervals containing a given point (stabbing query).

Where to use: Use for scheduling (finding overlapping meetings), network monitoring, and any interval stabbing query. Augmented BST with max endpoint.

```
dsa::interval_tree<int> it;
it.insert(dsa::interval_tree<int>::interval{1, 5, 0});
it.insert(dsa::interval_tree<int>::interval{3, 8, 1});
auto hits = it.query(4); // both intervals contain 4
```

Spatial & Range Structures

k-d Tree

Header: code/include/kd_tree.h

Class: dsa::kd_tree

Multi-dimensional nearest-neighbor and range queries. Partitions space by alternating splitting dimensions.

Why: You need nearest-neighbor or range queries in multi-dimensional space.

Where to use: Use for k-NN in ML, GIS queries, collision detection in games, and particle simulation. Good for low-to-medium dimensions.

```
dsa::kd_tree<2, double> kd({{0,0,0},{1,1,1},{5,5,2}});  
auto [pt, dist] = kd.nearest({1, 1});  
auto near = kd.range_query({1, 1}, 2.0);
```

Quadtree

Header: code/include/quadtree.h

Class: dsa::quadtree

Recursive 2D spatial partitioning with NW/NE/SW/SE children.

Why: Your data is 2D and you need recursive spatial partitioning.

Where to use: Use for 2D collision detection, image compression, and geographic indexing. Good when points are uniformly distributed.

```
dsa::quadtree qt({0, 0, 100, 100});  
qt.insert(10, 20, 0);  
qt.insert(50, 60, 1);  
auto near = qt.range_query(10, 20, 5.0);
```

R-Tree

Header: code/include/r_tree.h

Class: dsa::r_tree

Spatial index for multi-dimensional bounding-box queries. Groups nearby objects using minimum bounding rectangles.

Why: You need to index multi-dimensional objects (not just points) with bounding-box queries.

Where to use: Use for spatial databases, map applications, and game spatial indexing. Groups nearby objects using minimum bounding rectangles.

```
dsa::r_tree<2> rt;  
rt.insert({{1, 2, 0}});  
rt.insert({{3, 4, 1}});  
auto pts = rt.range_query({{0,0},{5,5}});
```

Orthogonal Range Tree

Header: code/include/orthogonal_range_tree.h

Class: dsa::orthogonal_range_tree

Segment tree on x-axis, sorted-by-y at each node. Efficient for axis-aligned rectangular range queries.

Why: You need to report all points in an axis-aligned rectangular range.

Where to use: Use for windowing queries in computational geometry, VLSI checking, and D range reporting. Segment tree on x, sorted by y.

```
dsa::orthogonal_range_tree<int> ort({{1,2,0},{3,4,1},{5,6,2}});
auto pts = ort.query(1, 5, 2, 6); // rectangle query
```

Segment Tree (Lazy)

Header: code/include/segment_tree_lazy.h

Class: dsa::segment_tree_lazy

Segment tree with lazy propagation for range updates + range queries. Supports sum, min, or max with range add/set.

Why: You need BOTH range updates (add/set) AND range queries on an array.

Where to use: Use when you need $O(\log n)$ range update AND range query. Lazy propagation defers updates. Essential for competitive programming.

```
dsa::segment_tree_lazy stl({1, 2, 3, 4, 5});
stl.update(1, 3, 10); // add 10 to indices [1,3]
int s = stl.query(0, 4); // sum of all elements
```

Fenwick Tree (BIT)

Header: code/include/fenwick_segment.h

Classes: dsa::fenwick_tree, dsa::segment_tree, dsa::union_find

Binary Indexed Tree for prefix sums. Simpler to implement than segment trees with similar performance.

Why: You need prefix sums with point updates, and want simpler code than segment trees.

Where to use: Use for counting inversions, prefix sum queries, and 2D range sums. Simpler than segment trees with similar $O(\log n)$ performance.

```
dsa::fenwick_tree<int> ft(10);
ft.add(0, 5); ft.add(1, 3);
int p = ft.prefix_sum(1); // 8
int r = ft.range_sum(0, 1); // 8
```

Union-Find

Header: code/include/union_find.h

Class: dsa::union_find

Disjoint-set with path compression + union by size. The inverse Ackermann function $\alpha(n)$ is effectively constant.

Why: You need to efficiently merge sets and check if two elements are in the same set.

Where to use: Use for Kruskal's MST, connected components in dynamic graphs, cycle detection, and percolation. Nearly $O(1)$ per operation.

```
dsa::union_find uf(5);
uf.unite(0, 1); // merge sets
uf.unite(1, 2);
uf.connected(0, 2); // true
uf.count(); // 3 components
```

van Emde Boas Tree

Header: code/include/veb_tree.h

Class: dsa::veb_tree

Near-constant time for integer operations when the universe U is bounded.

Why: You need $O(\log \log U)$ predecessor/successor on bounded integer keys.

Where to use: Use when keys are bounded integers and you need near-constant time operations. Expensive in space: $O(U)$ for universe size U .

```
dsa::veb_tree<256> veb;
veb.insert(5); veb.insert(3); veb.insert(7);
veb.successor(4); // 5
veb.predecessor(6); // 5
veb.erase(3);
```

X-Fast Trie

Header: code/include/x_fast_trie.h

Class: dsa::x_fast_trie

Trie-based structure achieving van Emde Boas-like performance with less space.

Why: You want vEB-like $O(\log \log U)$ performance with less space.

Where to use: Use as a space-efficient alternative to van Emde Boas trees for integer predecessor/successor queries.

```
dsa::x_fast_trie<256> xft;
xft.insert(5); xft.insert(3); xft.insert(7);
xft.successor(4); // 5
xft.predecessor(6); // 5
xft.erase(3);
```

Merkle Tree

Header: code/include/merkle_tree.h

Class: dsa::merkle_tree

Hash tree for data integrity verification. Each leaf hashes a data block; each internal node hashes its children.

Why: You need to verify data integrity efficiently across distributed systems.

Where to use: Use for blockchain, distributed databases, Git, and any system requiring tamper-evident data verification. $O(\log n)$ proof per block.

Bloom Filter

Header: code/include/bloom_filter.h

Class: dsa::bloom_filter

Probabilistic set membership. Space-efficient with tunable false-positive rate. No false negatives.

Why: You need fast set membership testing with low memory, and can tolerate false positives.

Where to use: Use when you need to filter out definitely-not-present items quickly: URL blacklists, database bloom filters, cache filtering. Never has false negatives.

```
dsa::bloom_filter bf(1000, 5);
bf.add("hello");
bf.might_contain("hello"); // true
bf.might_contain("world"); // maybe false
```

Part III — Algorithm Design Paradigms

Sorting

Header: code/include/sorting.h

12 sorting algorithms covering comparison-based, non-comparison, and specialized approaches.

Insertion Sort

Function: dsa::insertion_sort(span<T> a)

Builds the sorted array one element at a time by inserting each element into its correct position among the already-sorted prefix.

Why: $O(n^2)$ is acceptable for small n , or data is nearly sorted ($O(n)$ best case).

Where to use: Use for small arrays ($n \leq 20$), nearly-sorted data, as the base case in hybrid sorts, and online sorting. Best when data is almost sorted.

```
std::vector<int> a = {5, 3, 1, 4, 2};
dsa::insertion_sort(a);
// a = {1, 2, 3, 4, 5}
```

Merge Sort

Function: dsa::merge_sort(span<T> a)

Divide-and-conquer: split in half, recursively sort each half, merge. Stable sort with guaranteed $O(n \log n)$.

Why: You need guaranteed $O(n \log n)$ and/or stable sort, and can afford $O(n)$ extra space.

Where to use: Use when you need stability (preserving relative order of equal elements), guaranteed $O(n \log n)$, or external sorting (data larger than memory).

```
std::vector<int> a = {5, 3, 1, 4, 2};
dsa::merge_sort(a);
// a = {1, 2, 3, 4, 5}
```

Quick Sort

Function: `dsa::quick_sort(span<T> a)`

Partition around a pivot, recursively sort partitions. Fastest in practice despite worse-case.

Why: You want the fastest average-case sort and can accept $O(n^2)$ worst case (mitigated by randomization).

Where to use: Use as the default in-memory sort. Fastest in practice due to cache-friendly partitioning. Randomized pivot makes $O(n^2)$ extremely unlikely.

```
std::vector<int> a = {5, 3, 1, 4, 2};
dsa::quick_sort(a);
// a = {1, 2, 3, 4, 5}
```

3-Way Quick Sort

Function: `dsa::quick_sort_3way(span<T> a)`

Dutch National Flag partition: three-way split into less-than, equal-to, and greater-than pivot.

Why: Your data has many duplicate keys and standard quick sort degrades to $O(n^2)$.

Where to use: Use when data has many duplicates: database sorting, histograms, and categorical data. Three-way partition avoids $O(n^2)$ on equal keys.

```
std::vector<int> a = {5, 3, 3, 4, 3};
dsa::quick_sort_3way(a);
// a = {3, 3, 3, 4, 5}
```

Quickselect

Function: `dsa::quick_select(span<T> a, size_t k)`

Finds the k -th smallest element without fully sorting.

Why: You need the k -th smallest element without fully sorting the array.

Where to use: Use for finding medians, top- k queries, percentiles, and order statistics. $O(n)$ average without modifying the input order.

```
std::vector<int> a = {5, 3, 1, 4, 2};
int median = dsa::quick_select(a, 2); // 3rd smallest = 3
```

Shell Sort

Function: `dsa::shell_sort(span<T> a)`

Generalization of insertion sort using decreasing gap sequences. Elements far apart are sorted first, then reduced.

Why: You need a simple in-place sort with better than $O(n^2)$ performance and minimal code.

Where to use: Use in embedded systems, medium arrays ($n \leq 1000$), and when code simplicity matters more than optimal performance.

```
std::vector<int> a = {5, 3, 1, 4, 2};
dsa::shell_sort(a);
// a = {1, 2, 3, 4, 5}
```

Bubble Sort

Function: `dsa::bubble_sort(span<T> a)`

Repeatedly swap adjacent out-of-order elements. Stops early if no swaps occur.

Why: You need the simplest possible sort, or want to detect if data is already sorted.

Where to use: Use for teaching, detecting sorted arrays ($O(n)$ best case), and tiny datasets. Never use in production for large data.

```
std::vector<int> a = {5, 3, 1, 4, 2};
dsa::bubble_sort(a);
// a = {1, 2, 3, 4, 5}
```

Bottom-Up Merge Sort

Function: `dsa::merge_sort_bottom_up(span<T> a)`

Iterative merge sort avoiding recursion overhead. Merges subarrays of increasing size.

Why: You need merge sort without recursion overhead.

Where to use: Use in embedded systems where recursion is costly, and for linked list sorting without random access.

```
std::vector<int> a = {5, 3, 1, 4, 2};
dsa::merge_sort_bottom_up(a);
// a = {1, 2, 3, 4, 5}
```

Counting Sort

Function: `dsa::counting_sort(span<int> a)`

Non-comparison sort using an auxiliary count array.

Why: Your data consists of integers with a known, small range.

Where to use: Use when integer keys have small range ($k \ll n \log n$). $O(n + k)$ beats comparison sorts. Also used as subroutine in radix sort.

```
std::vector<int> a = {4, 2, 5, 2, 1};
dsa::counting_sort(a);
// a = {1, 2, 2, 4, 5}
```

Radix Sort (LSD)

Function: `dsa::radix_sort(span<int> a)`

Least-significant-digit first. Sorts by each digit position using counting sort.

Why: You're sorting fixed-length integers and want $O(d \cdot n)$ performance.

Where to use: Use for fixed-length keys: phone numbers, SSNs, dates. $O(d \cdot n)$ where d = number of digits. Faster than comparison sorts for bounded data.

```
std::vector<int> a = {170, 45, 75, 90, 802};
dsa::radix_sort(a);
// a = {45, 75, 90, 170, 802}
```

Radix Sort (MSD for Strings)

Function: `dsa::radix_sort_msd(span<string> a)`

Most-significant-digit first, recursive. Naturally handles variable-length strings.

Why: You're sorting variable-length strings by their lexicographic order.

Where to use: Use for sorting strings: dictionaries, URLs, file paths. MSD radix sort naturally handles variable-length strings.

```
std::vector<std::string> a = {"banana", "apple", "cherry"};
dsa::radix_sort_msd(a);
// a = {"apple", "banana", "cherry"}
```

Bucket Sort

Function: `dsa::bucket_sort(span<float> a)`

Distributes elements into buckets, sorts each bucket, concatenates.

Why: Your data is uniformly distributed floating-point numbers in $[0,1)$.

Where to use: Use when data is uniformly distributed: floating-point numbers, histogram equalization. $O(n)$ average for uniform input.

```
std::vector<float> a = {0.7f, 0.1f, 0.4f, 0.2f};
dsa::bucket_sort(a);
// a = {0.1f, 0.2f, 0.4f, 0.7f}
```

Greedy Algorithms

Header: code/include/greedy.h

Fractional Knapsack

Function: dsa::fractional_knapsack(capacity, items)

Greedy by value-to-weight ratio. Takes fractions of items to maximize value within capacity.

Why: You can take fractions of items to maximize value (items are divisible).

Where to use: Use when items are divisible: bandwidth sharing, CPU time slicing, commodity trading. Greedy by value/weight ratio gives optimal solution.

```
std::vector<dsa::frac_item> items =
    {{10,60},{20,100},{30,120}};
double val = dsa::fractional_knapsack(50, items);
// val = 240.0
```

Huffman Coding

Function: dsa::huffman_encode(freq_table)

Builds an optimal prefix-free binary code tree. Frequent characters get shorter codes.

Why: You need optimal prefix-free encoding for known symbol frequencies.

Where to use: Use for entropy encoding in compression: ZIP, JPEG, H.264. Gives optimal prefix-free codes. Often combined with LZ77/LZ78.

```
std::vector<std::pair<char,int>> freq =
    {{ 'a', 5 }, { 'b', 9 }, { 'c', 12 } };
auto codes = dsa::huffman_encode(freq);
// codes: [ ('a', "00"), ('b', "01"), ('c', "1") ]
```

Divide & Conquer

Header: code/include/divide_conquer.h

Closest Pair of Points

Function: dsa::closest_pair(points)

Divides points by x-coordinate, recursively finds closest pair in each half, then checks the strip around the dividing line.

Why: You need to find the closest pair among n points in 2D/3D.

Where to use: Use for facility location, collision detection, and clustering. $O(n \log n)$ beats the naive $O(n^2)$ for large point sets.

```
std::vector<std::pair<double,double>> pts =
    {{0,0},{1,1},{5,5},{6,6}};
auto [d, p1, p2] = dsa::closest_pair(pts);
// d = 1.414... (distance between (0,0) and (1,1))
```

Strassen's Matrix Multiply

Function: `dsa::strassen_multiply(A, B)`

Reduces 8 recursive multiplications to 7 at the cost of extra additions. Asymptotically faster than the naive $O(n^3)$.

Why: You need to multiply large matrices faster than $O(n^3)$.

Where to use: Use for large matrix multiplication where $n \geq 64$. Reduces from 8 to 7 recursive multiplications. Practical for large matrices due to cache effects.

```
std::vector<std::vector<double>> A =
    {{1,2},{3,4}};
std::vector<std::vector<double>> B =
    {{5,6},{7,8}};
auto C = dsa::strassen_multiply(A, B);
// C = {{19,22},{43,50}}
```

Dynamic Programming

Header: `code/include/dynamic_programming.h`

0/1 Knapsack

Function: `dsa::knapsack_01(capacity, items)`

Classic DP: fill a 2D table where $dp[i][w]$ = best value using first i items with capacity w . Each item taken at most once.

Why: You have items with weights/values and a capacity constraint, and items can't be split.

Where to use: Use for budget optimization, cargo loading, and investment selection. $O(nW)$ pseudo-polynomial time. Use for small W or exact solutions needed.

```
std::vector<std::pair<int,int>> items =
    {{2,3},{3,4},{4,5},{5,6}};
int val = dsa::knapsack_01(8, items);
// val = 10
```


Longest Common Subsequence

Function: `dsa::lcs(a, b)`

Finds the longest subsequence present in both strings (not necessarily contiguous).

Why: You need the longest common subsequence (not substring) between two sequences.

Where to use: Use for diff tools (git diff), version control, DNA alignment, and plagiarism detection. Shows how two sequences differ.

```
auto s = dsa::lcs("ABCBADAB", "BDCAB");  
// s = "BCAB" or "BDAB"
```

Edit Distance

Function: `dsa::edit_distance(a, b)`

Minimum number of insertions, deletions, and substitutions to transform one string into another.

Why: You need the minimum number of edits to transform one string into another.

Where to use: Use for spell checking, DNA alignment, speech recognition, and plagiarism detection. Foundation for fuzzy matching.

```
int dist = dsa::edit_distance("kitten", "sitting");  
// dist = 3
```

Longest Increasing Subsequence

Function: `dsa::lis_length(a)`

Finds the length of the longest strictly increasing subsequence using patience sorting.

Why: You need the length of the longest strictly increasing subsequence.

Where to use: Use for Stock Span, patience sorting, and when you need to find the longest increasing pattern. $O(n \log n)$ with patience sorting.

```
std::vector<int> a = {10,9,2,5,3,7,101,18};  
auto len = dsa::lis_length(a);  
// len = 4 (subsequence: 2,3,7,101)
```

Backtracking

Header: `code/include/backtracking.h`

N-Queens

Function: `dsa::n_queens(n)`

Place N queens on an N x N board so no two attack each other. Uses column and diagonal tracking for efficient pruning.

Why: You need to find all valid configurations satisfying constraints (classic CSP).

Where to use: Use for constraint satisfaction, puzzle solving, circuit board layout, and as a teaching example for backtracking with pruning.

```
auto sols = dsa::n_queens(4);
// sols.size() = 2
// sols[0] = {".Q..","...Q","Q...","..Q."}
```

Subset Sum

Function: `dsa::subset_sum(nums, target)`

Determines if any subset of integers sums to a given target. Uses DP table (not pure backtracking).

Why: You need to determine if any subset sums to a target value.

Where to use: Use for partition problems, fair division, and cryptographic applications. $O(n \cdot \text{target})$ pseudo-polynomial time.

```
bool ok = dsa::subset_sum({3, 34, 4, 12, 5, 2}, 9);
// ok = true (4 + 5 = 9)
```

Traveling Salesman

Function: `dsa::tsp(dist)`

Exact TSP solution via bitmask dynamic programming. Visits all cities exactly once and returns to start.

Why: You need the optimal tour visiting all cities exactly once (small n).

Where to use: Use for small instances ($n \leq 20$) of routing problems. For larger instances, use heuristics (nearest neighbor, 2-opt, genetic algorithms).

```
std::vector<std::vector<int>>> d =
    {{0,10,15,20},{10,0,35,25},
     {15,35,0,30},{20,25,30,0}};
int cost = dsa::tsp(d);
// cost = 80
```

Probabilistic Structures

Header: `code/include/probabilistic.h`

Reservoir Sampling

Function: `dsa::reservoir_sample(k, stream)`

Selects k items uniformly at random from a stream of unknown length. Each item has equal probability k/n of being selected.

Why: You need k random items from a stream of unknown length in one pass.

Where to use: Use for streaming random sampling, database sampling, A/B testing, and Monte Carlo. Each item has equal probability k/n .

```
std::vector<int> stream = {1,2,3,4,5,6,7,8,9,10};
auto sample = dsa::reservoir_sample(3, stream);
// sample.size() == 3, uniformly random
```

HyperLogLog

Class: `dsa::hyperloglog(precision)`

Probabilistic cardinality estimator. Uses the observation that hash values with many leading zeros are rare.

Why: You need to count distinct elements in a stream with sublinear space.

Where to use: Use for unique visitor counting, network flow monitoring, and SELECT DISTINCT estimation. $O(1)$ per element, tunable accuracy.

```
dsa::hyperloglog hll(10);
hll.add("hello"); hll.add("world");
hll.add("hello"); // duplicate
hll.count(); // ~2 (approximate cardinality)
```

Count-Min Sketch

Class: `dsa::count_min_sketch(width, depth)`

Probabilistic frequency estimator. Over-estimates but never under-estimates counts.

Why: You need to estimate frequencies in a stream, and can tolerate over-estimation.

Where to use: Use for network flow counting, stream frequency estimation, and heavy-hitter detection. Never under-estimates, may over-estimate.

```
dsa::count_min_sketch cms(1024, 5);
cms.add("hello", 3);
cms.add("world", 2);
cms.estimate("hello"); // >= 3
```

Part IV — Strings & Specialized Topics

Exact String Matching

Header: `code/include/exact_matching.h`

Five pattern matching algorithms. All return `vector<size_t>` of match start positions.

Morris-Pratt

Function: `dsa::morris_pratt_search(text, pattern)`

Precomputes prefix function to avoid redundant comparisons. The text pointer never moves backward.

Why: You need $O(n+m)$ pattern matching with simple prefix function.

Where to use: Use as the conceptual stepping stone to KMP. Text pointer never moves backward. Simpler than KMP but skips fewer characters.

```
auto pos = dsa::morris_pratt_search(
    "ABABDABACDABABCABAB", "ABABCABAB");
// pos = {9}
```

Knuth-Morris-Pratt (KMP)

Function: `dsa::kmp_search(text, pattern)`

Optimized prefix function uses pattern lookahead to skip more characters on mismatch.

Why: You need the standard $O(n+m)$ pattern matching algorithm.

Where to use: Use as the default linear-time pattern matcher. KMP's prefix function skips more characters than Morris-Pratt on mismatch.

```
auto pos = dsa::kmp_search(
    "ABABDABACDABABCABAB", "ABABCABAB");
// pos = {9}
```

Karp-Rabin

Function: `dsa::karp_rabin(pattern, base, mod)`

Uses rolling hash to compare pattern against text windows in $O(1)$. Only verifies on hash match.

Why: You want average-case $O(n+m)$ with simple rolling hash, and can accept $O(nm)$ worst case.

Where to use: Use for plagiarism detection, multiple pattern search, and when simplicity matters. Rolling hash gives $O(1)$ window comparison.

```
auto matches = dsa::karp_rabin("ABCABC", "ABC");
// matches = {0, 3}
```

Galil-Seiferas

Function: `dsa::galil_seiferas_search(text, pattern)`

Uses critical factorization for linear-time matching with skip optimization.

Why: You need optimal $O(n+m)$ matching with skip optimization via critical factorization.

Where to use: Use for long repeated patterns where skip optimization matters. Theoretically optimal but rarely used in practice.

```
auto pos = dsa::galil_seiferas_search("ABCABC", "ABC");  
// pos = {0, 3}
```

Boyer-Moore-Horspool

Function: `dsa::boyer_moore_horspool(pattern)`

Simplified Boyer-Moore using only bad-character shift. Compares right-to-left, skips large portions of text.

Why: You want the fastest practical string search for large alphabets.

Where to use: Use as the default practical string search. Bad-character shift skips large portions of text. Sublinear average-case.

```
dsa::boyer_moore_horspool bm("needle");  
auto pos = bm.search("haystack needle here");  
// pos = {8}
```

String Search & Pattern Matching

Header: `code/include/string_search.h`

Z-Algorithm

Function: `dsa::z_function(s)`

For each position i , computes $z[i]$ = length of longest substring starting at i that matches the prefix.

Why: Pattern matching (concatenate pattern + \$ + text, find z-values equal to pattern length), shortest unique substring, and run-length detection.

```
auto z = dsa::z_function("aabxaab");  
auto pos = dsa::z_search("aabxaabxcaab", "aab");  
// pos = {0, 9}
```

Boyer-Moore (Good Suffix)

Function: `dsa::boyer_moore(text, pattern)`

Uses both bad-character and good-suffix heuristics for maximum skip distance.

Why: You want the fastest practical string search using both bad-character and good-suffix heuristics.

Where to use: Use as the fastest practical string search. Both heuristics give maximum skip distance. Sublinear average-case.

```
dsa::boyer_moore bm("needle");
auto pos = bm.search("haystack needle here");
// pos = {8}
```

Manacher's Palindrome

Function: `dsa::manacher(s)`

Finds all odd-length palindromes centered at each position in linear time using symmetry exploitation.

Why: You need to find all palindromic substrings in linear time.

Where to use: Use for longest palindromic substring, palindrome detection in DNA, and text compression. $O(n)$ time exploits palindrome symmetry.

```
auto lps = dsa::longest_palindrome("babad");
// lps = "bab" or "aba"
```

Approximate String Matching

Header: `code/include/approximate_matching.h`

Levenshtein Distance

Function: `dsa::levenshtein(a, b)`

Minimum edit distance: insertions, deletions, and substitutions.

Why: You need the minimum edit distance (insertions + deletions + substitutions).

Where to use: Use for spell checking, DNA alignment, speech recognition, and record deduplication. The standard edit distance metric.

```
int dist = dsa::levenshtein("kitten", "sitting");
// dist = 3
```

Hamming Distance

Function: `dsa::hamming_distance(a, b)`

Number of positions where corresponding characters differ. Only defined for equal-length strings.

Why: You need to count positions where two equal-length strings differ.

Where to use: Use for fixed-length string comparison: error-correcting codes, hash comparison, DNA mutation counting. Only for equal-length strings.

```
auto dist = dsa::hamming_distance("karolin", "kathrin");
// dist = 3
```

Myers' Bit-Parallel

Function: `dsa::myers_bit_parallel(text, pattern, threshold)`

Simulates the DP edit distance matrix using bit operations. Extremely fast for small patterns and thresholds.

Why: You need approximate pattern matching with small edit distance threshold.

Where to use: Use for bioinformatics alignment, spell checking, and approximate grep. Bit-parallel simulation is extremely fast for small patterns.

```
auto pos = dsa::myers_approximate(
    "hello world", "helo", 1);
// pos = {0} (distance 1: 'l' missing)
```

Shift-Or (Approximate)

Function: `dsa::shift_or_approximate(text, pattern, threshold)`

Bit-parallel approximate matching using the Shift-Or framework.

Why: You need approximate pattern matching using bit-parallel DP.

Where to use: Use for fast approximate matching with Hamming distance. Bit-parallel approach processes multiple positions simultaneously.

```
auto pos = dsa::shift_or_approximate(
    "hello world", "helo", 1);
// pos = {0}
```

Multiple Pattern Matching

Header: `code/include/multiple_matching.h`

Aho-Corasick

Function: `dsa::aho_corasick_search(text, patterns)`

Builds an automaton from all patterns simultaneously. Scans text once, finding all pattern occurrences.

Why: You need to find ALL occurrences of MULTIPLE patterns in a single text scan.

Where to use: Use for multi-pattern matching: antivirus scanning, intrusion detection, bioinformatics. Builds one automaton from all patterns.

```
dsa::aho_corasick ac({"he", "she", "his", "hers"});
auto matches = ac.search("ahishers");
// matches: (1,"his"), (1,"he"), (4,"she"), (4,"he"), (4,"hers")
```

Commentz-Walter

Function: `dsa::commentz_walter_search(text, patterns)`

Multi-pattern search combining Shift-Or with Aho-Corasick concepts.

Why: You need multi-pattern search with variable-length patterns.

Where to use: Use as an alternative to Aho-Corasick when pattern lengths vary significantly. Combines Shift-Or with Aho-Corasick concepts.

```
dsa::commentz_walter cw({"he", "she", "his", "hers"});
auto matches = cw.search("ahishers");
// matches: (position, pattern_id) for each match
```

Text Decomposition

Header: `code/include/text_decomposition.h`

Lyndon Factorization

Function: `dsa::lyndon_factorization(s)`

Duval's algorithm decomposes a string into Lyndon words (lexicographically strictly smallest rotations).

Why: You need to decompose a string into Lyndon words.

Where to use: Use for minimal string rotation (Booth's), suffix array construction (DC3/SA-IS), and string periodicity analysis.

```
auto factors = dsa::lyndon_factorization("banana");
// factors: "b" "an" "an" "a" (Lyndon factorization)
bool is_ld = dsa::is_lyndon_word("an");
// is_ld = true
```

Critical Factorization

Function: `dsa::critical_factorization(s)`

Finds the critical index that enables optimal two-way string matching.

Why: You need the critical index for optimal two-way string matching.

Where to use: Use as preprocessing for Crochemore-Perrin two-way algorithm. Enables $O(n+m)$ matching with left-to-right text scan.

```
auto [pos, period] = dsa::critical_factorization("abcabc");
// pos = 3, period = 3
```


String Compression

Header: code/include/string_compression.h

Four classic compression algorithms with encode/decode pairs.

Run-Length Encoding

Functions: dsa::rle_encode(s), dsa::rle_decode(s)

Replaces consecutive repeated characters with character + count.

Why: Your data has long runs of repeated characters.

Where to use: Use for run-length data: fax, BMP images, sensor data. Simple but only effective when runs are long. Often combined with other methods.

```
auto enc = dsa::rle_encode("AAABBBCCD");  
// enc = "A3B3C2D1"  
auto dec = dsa::rle_decode(enc);  
// dec = "AAABBBCCD"
```

LZ77

Functions: dsa::lz77_encode(s), dsa::lz77_decode(tokens)

Sliding-window dictionary compression. References previous occurrences via (offset, length) tuples.

Why: You need sliding-window dictionary compression.

Where to use: Use as the foundation of DEFLATE (gzip, PNG, ZIP). References previous occurrences via (offset, length) tuples.

```
auto tokens = dsa::lz77_encode("ABCABCABC");  
auto orig = dsa::lz77_decode(tokens);  
// orig == "ABCABCABC"
```

LZ78

Functions: dsa::lz78_encode(s), dsa::lz78_decode(tokens)

Dictionary-based compression building an explicit dictionary during encoding.

Why: You need dictionary-based compression with explicit dictionary building.

Where to use: Use for LZW variant compression: GIF, Unix compress. Builds dictionary explicitly during encoding.

```
auto tokens = dsa::lz78_encode("TOBEORNOTTOBE");  
auto orig = dsa::lz78_decode(tokens);  
// orig == "TOBEORNOTTOBE"
```

LZW

Functions: `dsa::lzw_encode(s)`, `dsa::lzw_decode(codes)`

Adaptive dictionary compression. Dictionary is built implicitly during both encoding and decoding.

Why: You need adaptive dictionary compression where dictionary is built during both encode and decode.

Where to use: Use for GIF, TIFF, PDF compression. Dictionary is built implicitly, no need to transmit it separately.

```
auto codes = dsa::lzw_encode("TOBEORNOTTOBE");
auto orig = dsa::lzw_decode(codes);
// orig == "TOBEORNOTTOBE"
```

Burrows-Wheeler Transform

Header: `code/include/bwt.h`

Functions: `dsa::bwt_encode(s)`, `dsa::bwt_decode(s)`

Reorders a string so that similar characters cluster together, enabling high compression ratios.

Why: You need to reorder a string so similar characters cluster together for better compression.

Where to use: Use as preprocessing for bzip2 compression. Enables high compression ratios by clustering similar characters.

```
auto encoded = dsa::bwt_encode("banana");
auto decoded = dsa::bwt_decode(encoded);
// decoded == "banana"
```

FM-Index

Class: `dsa::fm_index`

Compressed full-text index using BWT. Supports substring search and counting without decompression.

Why: You need to search compressed text without decompressing.

Where to use: Use for compressed full-text search: genome search, bioinformatics. Supports substring search and counting on compressed data.

```
dsa::fm_index fm("banana");
auto count = fm.count("ana"); // 2
auto pos = fm.search("ana"); // {1, 3}
```

Suffix Array Search

Header: `code/include/suffix_array_search.h`

Kasai LCP

Function: `dsa::kasai_lcp(s, sa)`

Constructs the LCP (longest common prefix) array from a suffix array in linear time.

Why: You need the LCP array from a suffix array.

Where to use: Use for longest repeated substring, distinct substring counting, and as input to many string algorithms. $O(n)$ construction.

```
std::string s = "banana";
std::vector<size_t> sa = {5, 3, 1, 0, 4, 2};
auto lcp = dsa::kasai_lcp(s, sa);
// lcp = {-, 1, 3, 0, 0, 2} (lcp[0] undefined)
```

SA Search

Function: `dsa::sa_search(sa, s, pattern)`

Binary search on suffix array to find pattern occurrences.

Why: You need to find pattern occurrences using binary search on a suffix array.

Where to use: Use for full-text search on suffix arrays. $O(m \log n)$ per query. Simpler than suffix tree for static texts.

```
std::string s = "banana";
std::vector<size_t> sa = {5, 3, 1, 0, 4, 2};
auto pos = dsa::sa_search(sa, s, "ana");
// pos = {1, 3}
```

NFA Regex Matching

Header: `code/include/nfa.h`

Function: `dsa::nfa_match(pattern, text)`

Thompson's NFA construction for regular expression matching. Supports `*`, `+`, `?`, `—`, and `()`.

Why: You need regular expression matching with `*`, `+`, `?`, `—`, and `()`.

Where to use: Use for regex matching: input validation, log filtering, web scraping. Thompson's NFA construction gives $O(nm)$ matching.

```
bool ok = dsa::nfa_match("(a|b)*abb", "aababb");
// ok = true
```

Graph Algorithms

Header: `code/include/graph.h`

Class: `dsa::graph<Weight>`

Adjacency-list graph with both directed and undirected edge support.

DFS / BFS

Functions: `g.dfs(start, visitor)`, `g.bfs(start, visitor)`

DFS uses a stack (recursive); BFS uses a queue. Both visit every reachable vertex exactly once.

Applications (DFS): Topological sorting, cycle detection, connected components, maze solving, path finding in puzzles, and backtracking search.

Applications (BFS): Shortest paths in unweighted graphs, level-order tree traversal, peer-to-peer network discovery, web crawlers, and social network degree-of-separation.

```
dsa::graph<int> g(4);
g.add_edge(0, 1); g.add_edge(1, 2); g.add_edge(2, 3);
g.dfs(0, [](int v) { /* visit v */ });
g.bfs(0, [](int v) { /* visit v */ });
```

Connected Components

Function: `g.connected_components()`

Assigns a component ID to each vertex using BFS. Vertices in the same component are mutually reachable.

Why: You need to find all connected components in an undirected graph.

Where to use: Use for friend group analysis, image processing, network reliability, and dependency analysis. Assigns component IDs via BFS.

```
dsa::graph<int> g(4);
g.add_edge(0, 1); g.add_edge(2, 3);
auto cc = g.connected_components();
// cc = {0,0,1,1} (two components)
```

Dijkstra

Function: `g.dijkstra(start)`

Finds shortest paths from a source vertex with non-negative edge weights using a min-priority queue.

Why: You need shortest paths from a source with non-negative edge weights.

Where to use: Use for GPS, IP routing, map applications, and robot path planning. $O((V+E) \log V)$ with priority queue.

```
dsa::graph<int> g(5);
g.add_edge(0, 1, 10);
g.add_edge(1, 3, 10);
auto dist = g.dijkstra(0);
// dist[3] = 20
```

Bellman-Ford

Function: `g.bellman_ford(start)`

Shortest paths from source, handling negative edge weights. Detects negative cycles.

Why: You need shortest paths and the graph may have negative edge weights.

Where to use: Use when negative edges exist: arbitrage detection, currency exchange. Detects negative cycles. $O(VE)$ time.

```
dsa::graph<int> g(3);
g.add_edge(0, 1, 4); g.add_edge(1, 2, -2);
auto dist = g.bellman_ford(0);
// dist = {0, 4, 2}
```

Floyd-Warshall

Function: `g.floyd_warshall()`

All-pairs shortest paths via dynamic programming. Simple triple-nested loop.

Why: You need all-pairs shortest paths in a dense graph.

Where to use: Use for all-pairs shortest paths in dense graphs, transitive closure, and small road networks. $O(V^3)$ simple DP.

```
dsa::graph<int> g(3);
g.add_edge(0, 1, 3); g.add_edge(1, 2, 1);
auto all = g.floyd_warshall();
// all[0][2] = 4
```

Topological Sort

Function: `g.topological_sort()`

Linear ordering of vertices in a DAG such that every edge goes from earlier to later.

Why: You need a linear ordering of vertices in a DAG respecting edge constraints.

Where to use: Use for dependency resolution: build systems, course prerequisites, task scheduling. $O(V+E)$ topological sort.

```
dsa::graph<int> g(4);
g.add_edge(0, 1); g.add_edge(0, 2); g.add_edge(1, 3);
auto order = g.topological_sort();
// order = {0, 1, 2, 3} or {0, 2, 1, 3}
```

Cycle Detection

Function: `g.has_cycle()`

DFS-based detection using three coloring states (Unvisited, InStack, Done).

Why: You need to detect if a directed graph contains a cycle.

Where to use: Use for deadlock detection, circular dependency checking, and infinite loop detection. DFS-based with three coloring states.

```
dsa::graph<int> g(3);
g.add_edge(0, 1); g.add_edge(1, 2);
g.has_cycle(); // false
g.add_edge(2, 0);
g.has_cycle(); // true
```

Bipartiteness Check

Function: `g.is_bipartite()`

2-coloring via BFS. Returns the coloring or empty if the graph is not bipartite.

Why: You need to 2-color a graph or detect odd-length cycles.

Where to use: Use for matching problems, graph coloring lower bounds, and bipartiteness checking. BFS-based 2-coloring.

```
dsa::graph<int> g(4);
g.add_edge(0, 1); g.add_edge(1, 2); g.add_edge(2, 3);
auto color = g.is_bipartite();
// color = {0, 1, 0, 1} (bipartite)
```

Transitive Closure

Function: `g.transitive_closure()`

Boolean reachability matrix: `reach[i][j]` is true iff `j` is reachable from `i`.

Why: You need to know if vertex `j` is reachable from vertex `i`.

Where to use: Use for reachability queries, dependency analysis, and type hierarchy checking. $O(V^3)$ Boolean matrix.

```
dsa::graph<int> g(3);
g.add_edge(0, 1); g.add_edge(1, 2);
auto tc = g.transitive_closure();
// tc[0][2] = true
```

DAG Shortest Path

Function: `g.dag_shortest_path(start)`

Processes vertices in topological order for optimal single-pass shortest path.

Why: You need shortest paths in a DAG efficiently.

Where to use: Use for project scheduling, critical path, and DAG optimization. $O(V+E)$ single pass in topological order.

```
dsa::graph<int> g(4);
g.add_edge(0, 1, 1); g.add_edge(0, 2, 4);
g.add_edge(1, 3, 2); g.add_edge(2, 3, 1);
auto dist = g.dag_shortest_path(0);
// dist = {0, 1, 4, 3}
```

Eulerian Cycle

Function: `g.eulerian_cycle()`

Hierholzer's algorithm for finding a cycle that uses every edge exactly once. Requires in-degree = out-degree for all vertices.

Why: You need a cycle that uses every edge exactly once.

Where to use: Use for genome assembly, Chinese postman problem, and graph drawing. Requires in-degree = out-degree for all vertices.

```
dsa::graph<int> g(3);
g.add_edge(0, 1); g.add_edge(1, 2); g.add_edge(2, 0);
auto cycle = g.eulerian_cycle();
// cycle = {0, 1, 2, 0} (or rotation)
```

Minimum Spanning Tree

Kruskal's MST

Header: `code/include/graph.h`

Function: `dsa::kruskal_mst(n, edges)`

Sorts edges by weight, greedily adds cheapest edges that don't form cycles (using Union-Find).

Why: You need the minimum weight spanning tree using greedy edge selection.

Where to use: Use for network design, clustering, and Steiner tree approximation. Sort edges greedily with Union-Find for cycle detection.

```
std::vector<dsa::weighted_edge> edges =
    {{0,1,4},{1,2,2},{0,2,3}};
auto mst = dsa::kruskal_mst(3, edges);
// mst has 2 edges, total weight = 5
```

Boruvka's MST

Header: `code/include/graph.h`

Function: `dsa::boruvka_mst(n, edges)`

Each component picks its cheapest outgoing edge. Parallelizable.

Why: You need a parallelizable MST algorithm.

Where to use: Use for parallel MST computation. Each component picks cheapest outgoing edge. Parallelizable unlike Prim's.

```
std::vector<dsa::weighted_edge> edges =
    {{0,1,4},{1,2,2},{0,2,3}};
auto mst = dsa::boruvka_mst(3, edges);
// mst has 2 edges, total weight = 5
```

Prim's MST

Header: code/include/prim.h

Function: dsa::prim(graph, source)

Grows a single tree from a source vertex by always adding the cheapest edge connecting the tree to a non-tree vertex.

Why: You need MST growing from a single source vertex.

Where to use: Use for network design, cluster analysis. Grows tree by always adding cheapest edge to non-tree vertex.

```
dsa::prim_mst pm(4);
pm.add_edge(0, 1, 4); pm.add_edge(1, 2, 2);
pm.add_edge(2, 3, 3); pm.add_edge(0, 3, 5);
auto res = pm.compute();
// res.total_weight = 9
```

Strongly Connected Components

Header: code/include/scc.h

Kosaraju's Algorithm

Function: dsa::kosaraju(graph)

Two-pass DFS: first pass gets finish order on original graph, second pass finds components on transpose.

Why: You need to find strongly connected components in a directed graph.

Where to use: Use for dependency analysis, web page clustering, and graph simplification. Two-pass DFS on original and transpose.

```
dsa::kosaraju_scc ks(5);
ks.add_edge(0, 1); ks.add_edge(1, 2);
ks.add_edge(2, 0); ks.add_edge(2, 3);
ks.add_edge(3, 4);
auto sccs = ks.find_sccs();
// sccs = {{0,1,2}, {3}, {4}}
```


Tarjan's Algorithm**Function:** `dsa::tarjan(graph)`

Single-pass DFS using a stack. Finds SCCs by tracking discovery times and low-link values.

Why: You need SCCs with a single DFS pass (no transpose graph needed).**Where to use:** Use for SCCs when you want single-pass simplicity. Uses discovery times and low-link values.

```
dsa::tarjan_scc ts(5);
ts.add_edge(0, 1); ts.add_edge(1, 2);
ts.add_edge(2, 0); ts.add_edge(2, 3);
ts.add_edge(3, 4);
auto sccs = ts.find_sccs();
// sccs = {{4}, {3}, {0,1,2}}
```

Maximum Flow & Matching***Dinic's Algorithm*****Header:** `code/include/dinic.h`**Class:** `dsa::dinic<FlowUnit>`

Builds level graphs via BFS, then finds blocking flows via DFS. Much faster in practice than Ford-Fulkerson on unit-capacity networks.

Why: You need maximum flow with good practical performance.**Where to use:** Use as the default max-flow algorithm. BFS level graphs + DFS blocking flows. Much faster than Ford-Fulkerson in practice.

```
dsa::dinic<int> d(4);
d.add_edge(0, 1, 10);
d.add_edge(0, 2, 10);
d.add_edge(1, 3, 10);
d.add_edge(2, 3, 10);
int flow = d.max_flow(0, 3);
// flow = 20
```

Ford-Fulkerson**Header:** `code/include/flow_matching.h`**Class:** `dsa::ford_fulkerson(n)`

Augments along augmenting paths until no more exist. Complexity depends on max flow value.

Why: You want the simplest augmenting-path max-flow algorithm.**Where to use:** Use for understanding max-flow concepts. Simple but $O(E * \text{max_flow})$ time. Use Dinic's or Edmonds-Karp for better bounds.

```
dsa::ford_ulkerson ff(4);
ff.add_edge(0, 1, 10); ff.add_edge(0, 2, 10);
ff.add_edge(1, 3, 10); ff.add_edge(2, 3, 10);
int flow = ff.max_flow(0, 3);
// flow = 20
```

Edmonds-Karp

Header: code/include/flow_matching.h

Class: dsa::edmonds_karp(n)

Ford-Fulkerson using BFS for shortest augmenting path. Guaranteed polynomial time.

Why: You want Ford-Fulkerson with guaranteed polynomial time.

Where to use: Use when you want BFS shortest augmenting path. $O(VE^2)$ guaranteed. Simpler than Dinic's for small graphs.

```
dsa::edmonds_karp ek(4);
ek.add_edge(0, 1, 10); ek.add_edge(0, 2, 10);
ek.add_edge(1, 3, 10); ek.add_edge(2, 3, 10);
int flow = ek.max_flow(0, 3);
// flow = 20
```

Hopcroft-Karp (Bipartite Matching)

Header: code/include/flow_matching.h

Class: dsa::bipartite_matching(n_left, n_right)

Finds maximum matching in bipartite graphs by finding multiple augmenting paths per BFS phase.

Why: You need maximum matching in a bipartite graph.

Where to use: Use for job assignment, resource allocation, and matching problems. $O(E \sqrt{V})$ via multiple augmenting paths per BFS phase.

```
dsa::bipartite_matching bm(3, 3);
bm.add_edge(0, 0);
bm.add_edge(0, 1);
bm.add_edge(1, 1);
bm.add_edge(2, 2);
int sz = bm.max_matching();
// sz = 3
```

Hungarian Algorithm

Header: code/include/hungarian.h

Function: dsa::hungarian(cost_matrix)

Finds minimum-cost perfect matching in a bipartite graph (assignment problem).

Why: You need minimum-cost perfect matching in a bipartite graph.

Where to use: Use for assignment problems: workers to tasks, exam seating, ride-sharing. $O(n^3)$ optimal solution.

```
std::vector<std::vector<int>>> cost =
    {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
auto res = dsa::hungarian_min(cost);
// res.cost = 15, res.assignment = {0,1,2}
```

Additional Algorithms

de Bruijn Graph & Sequence

Header: code/include/de_bruijn.h

Function: dsa::de_bruijn_sequence(k, n)

Generates a cyclic sequence where every possible substring of length n over an alphabet of size k appears exactly once.

Why: You need a cyclic sequence containing every possible substring of length n.

Where to use: Use for genome assembly, lock combinations, and pseudo-random number generation. Every n-length substring appears exactly once.

```
auto seq = dsa::de_bruijn_sequence(2, 3);
// seq contains all 3-bit binary substrings
```

Landau-Vishkin

Header: code/include/additional_algorithms.h

Function: dsa::landau_vishkin(a, b, k)

Computes edit distance only up to threshold k. Returns early if distance exceeds k.

Why: You need edit distance only up to threshold k, and want early termination.

Where to use: Use for spelling correction, near-duplicate detection, and DNA comparison when you only care about small differences.

```
auto matches = dsa::landau_vishkin(
    "hello world", "helo word", 3);
// matches = {positions within edit distance 3}
```

Hirschberg LCS

Header: code/include/additional_algorithms.h

Function: dsa::hirschberg_lcs(a, b)

Space-efficient LCS computation. Divides and conquers using linear-space needleman-wunsch.

Why: You need LCS with $O(\min(m,n))$ space instead of $O(mn)$.

Where to use: Use for sequence alignment on memory-constrained devices, diff algorithms, and bioinformatics where space is limited.

```
auto lcs = dsa::hirschberg_lcs("ABCBADAB", "BDCAB");  
// lcs = "BCAB" (O(min(m,n)) space)
```

Shortest Common Superstring

Header: code/include/additional_algorithms.h

Function: dsa::shortest_common_superstring(strings)

Finds the shortest string containing all input strings as substrings.

Why: You need the shortest string containing all input strings as substrings.

Where to use: Use for genome assembly from overlapping fragments and text reconstruction. Greedy approximation.

```
auto scs = dsa::shortest_common_superstring(  
    {"ABC", "BCA", "CAB"});  
// scs = "ABCAB" (shortest containing all)
```

Crochemore-Perrin Two-Way

Header: code/include/additional_algorithms.h

Function: dsa::crochemore_perrin(text, pattern)

Optimal two-way pattern matching using critical factorization. Text is scanned left-to-right.

Why: You need optimal $O(n+m)$ two-way pattern matching.

Where to use: Use for optimal string matching with left-to-right text scan. Critical factorization enables $O(n+m)$ with minimal preprocessing.

```
auto pos = dsa::crochemore_perrin(  
    "ABCABCABC", "ABCABC");  
// pos = {0, 3}
```

Quick Reference

Header	Category	Complexity
sorting.h	Sorting (12 algos)	$O(n \log n)$ avg
graph.h	Graph (15+ algos)	$O(V+E)$ – $O(V^3)$
dynamic_programming.h	DP (4 algos)	$O(nW)$ – $O(mn)$
greedy.h	Greedy (2 algos)	$O(n \log n)$
divide_conquer.h	D&C (2 algos)	$O(n \log n)$ – $O(n^{2.8})$
backtracking.h	Backtracking (3)	exponential
exact_matching.h	String (5 algos)	$O(n+m)$
approximate_matching.h	String (4)	$O(nm)$
multiple_matching.h	String (2)	$O(n+m+z)$
string_compression.h	Compression (4)	$O(n)$
dinic.h	Max Flow	$O(V^2 E)$
flow_matching.h	Flow/Match (3)	$O(VE^2)$
probabilistic.h	Probabilistic (3)	$O(1)/\text{elem}$
bloom_filter.h	Probabilistic	$O(k)/\text{elem}$
avl_tree.h	Balanced BST	$O(\log n)$
red_black_tree.h	Balanced BST	$O(\log n)$
b_tree.h	Multi-way tree	$O(\log n)$
fenwick_segment.h	Range query	$O(\log n)$
segment_tree_lazy.h	Range query	$O(\log n)$
union_find.h	Disjoint set	$O(\alpha(n))$
kd_tree.h	Spatial	$O(\sqrt{n})$ NN
bwt.h	BWT/FM-index	$O(n \log n)$
nfa.h	Regex	$O(nm)$

*All implementations live in namespace dsa, use C++23 features, and are header-only.
Build: `cd code && cmake -B build && cmake --build build && ctest --test-dir build`*